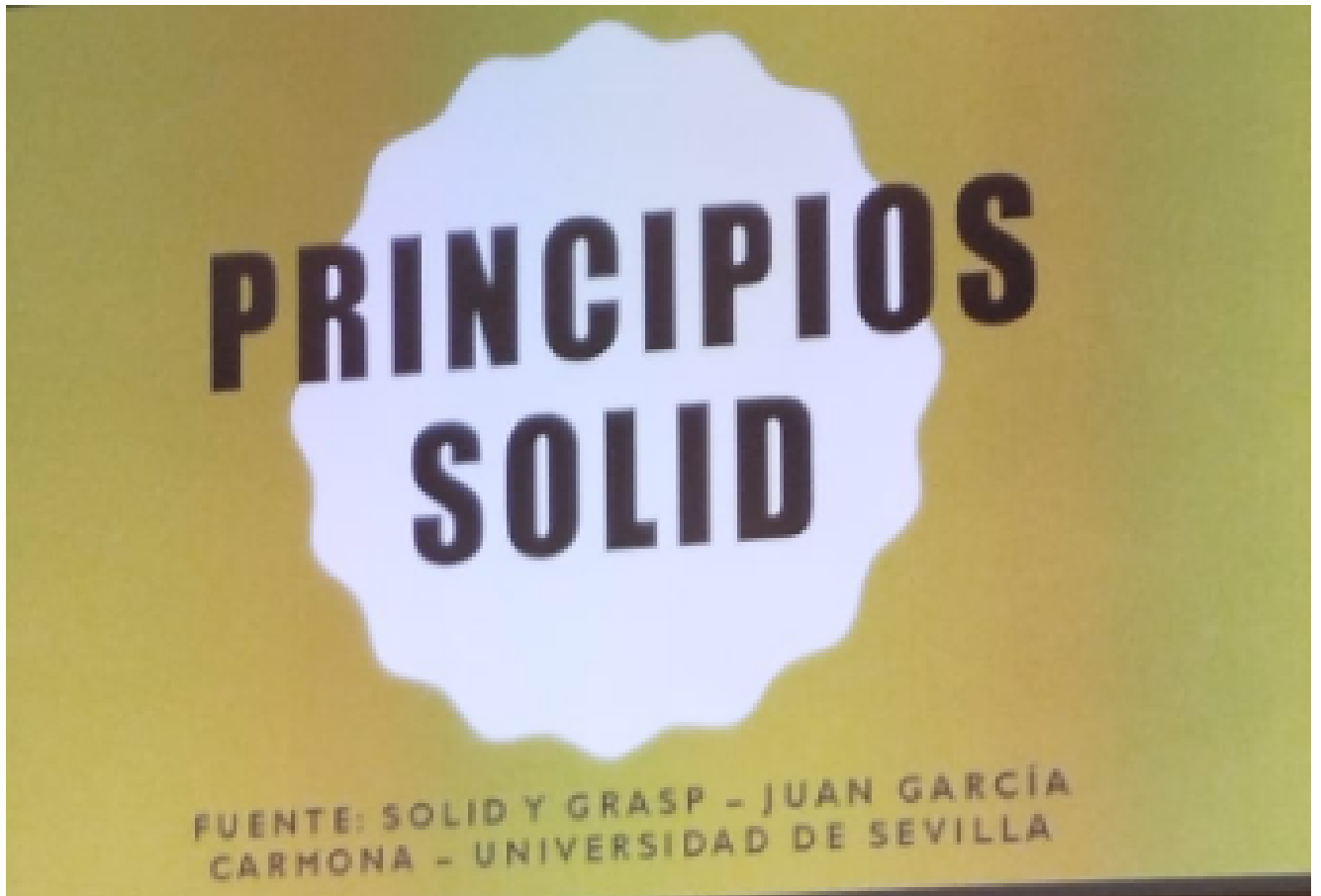


CLASE 1 - 13/08

SOLID

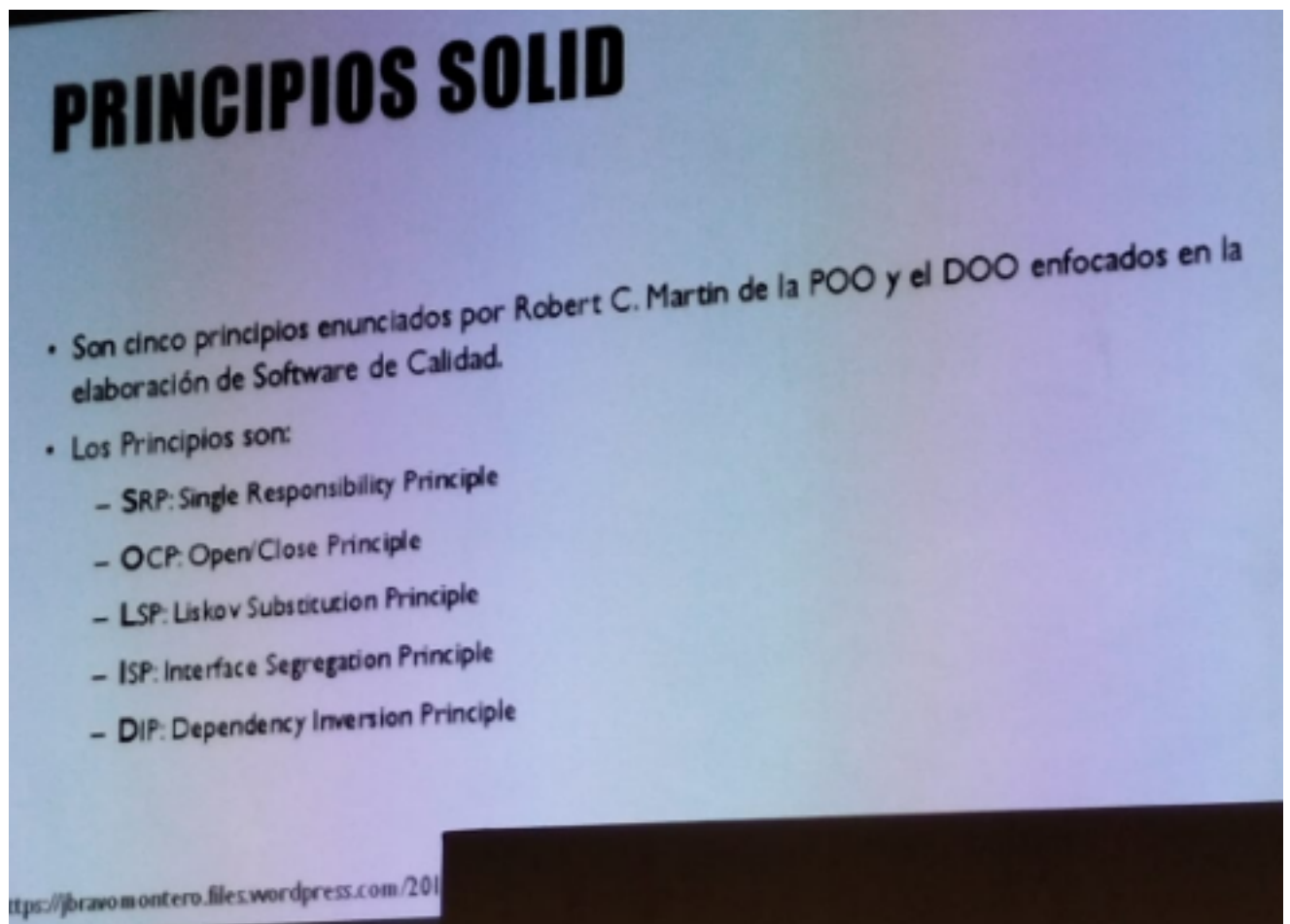


Para este tema hay un apunte que mostró el profe y sino una página que incluye un video también.

<https://devexpert.io/blog/principios-solid>

  Principios SOLID - 5 + 1 Reglas que CAMBIARÁN tu forma de PROGRAMAR
las imágenes con el fondo celeste saque del video que está ahí

PRINCIPIOS SOLID



-estos principios no es que sean muy novedosos, sino que el aporte que hizo es juntarlos, ponerles un acrónimo y los adapto a la programación. Son principios que existían hace mucho pero el lo que hizo fue juntarlos

1. Software más flexible: mejoran la cohesión disminuyendo el acoplamiento

2. Te van a hacer entender mucho mejor las arquitecturas

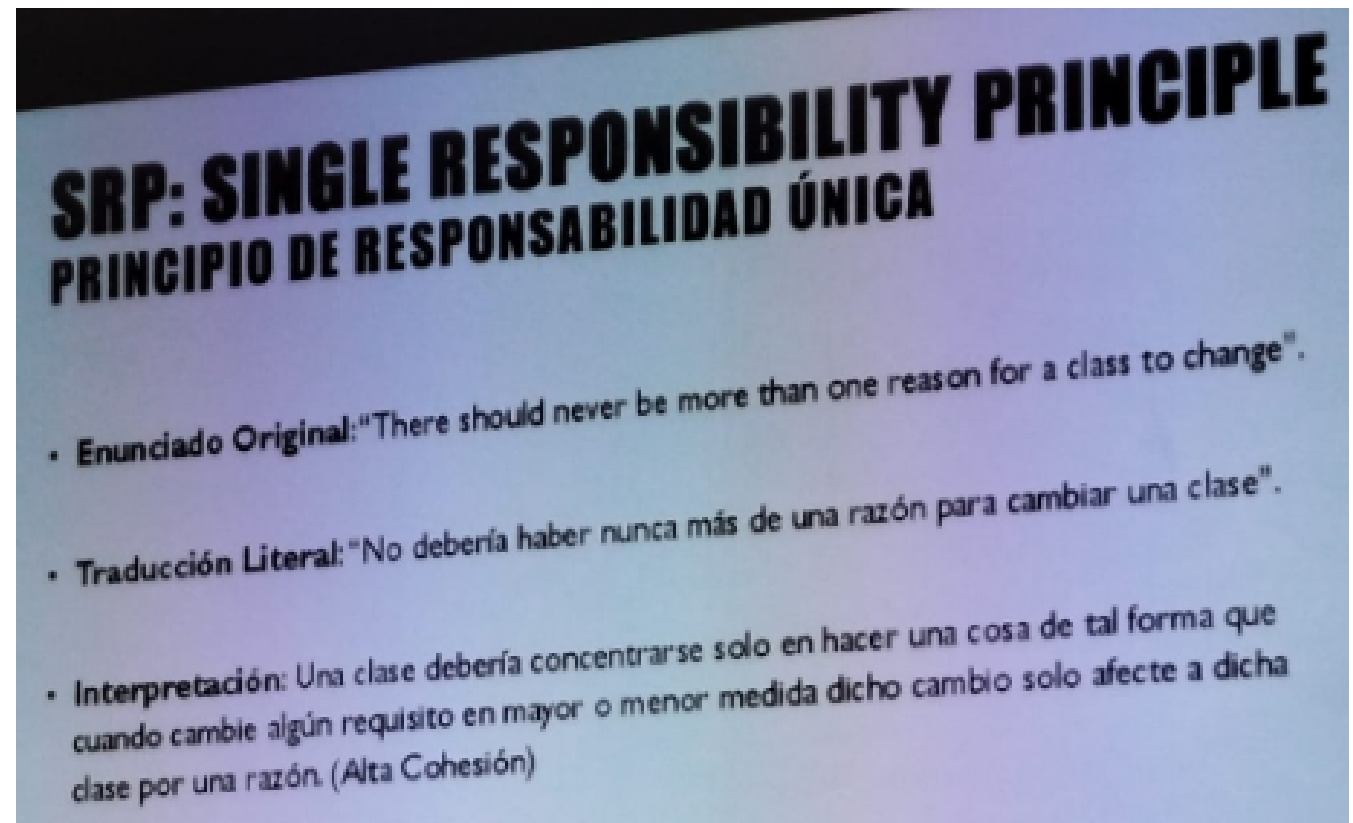
3. Simplifican la creación de tests

Unos principios no pueden existir sin los otros

Al cumplir un Principio puede que estés incumpliendo otro

SRP - Single Responsibility Principle

Responsabilidad única



-intenta prevenir la propagación de cambios muy extensa

-se trata de minimizar cambios

-el peligro de este principio es tener muchas clases de una sola línea

-se trata de hacer una clase cohesiva

-si tiene muchos métodos

-analizando que los métodos utilizan una mitad y otros métodos usan otra mitad de la clase, entonces la clase pueda dividirse en dos

(es diferente al concepto de responsabilidad que venimos viendo)

Como darnos cuenta?

En una misma clase están involucradas dos capas de la arquitectura

El número de métodos públicos

Los métodos que usan cada uno de los campos de esa clase

Nos cuesta testear la clase

Cada vez que escribes una nueva funcionalidad, esa clase se ve afectada

Por el número de líneas

-No hay regla de oro para estar 100% seguros, la práctica nos va a ir haciendo darnos cuenta

OCP - Open / Closed Principle

Abierto Cerrado

-evita tener que testear una clase ya probada

-abierto a extensión, cerrado a modificaciones

El principio Open/Closed se suele resolver utilizando polimorfismo

¿Cómo detectar que estamos violando el principio Open/Closed?

Intentar hacer un código 100% Open/Closed es prácticamente imposible

-intentar hacerlo Open/Close a veces puede llegar hacerlo mucho más complejo, sólo limitarse a cuando surge la necesidad

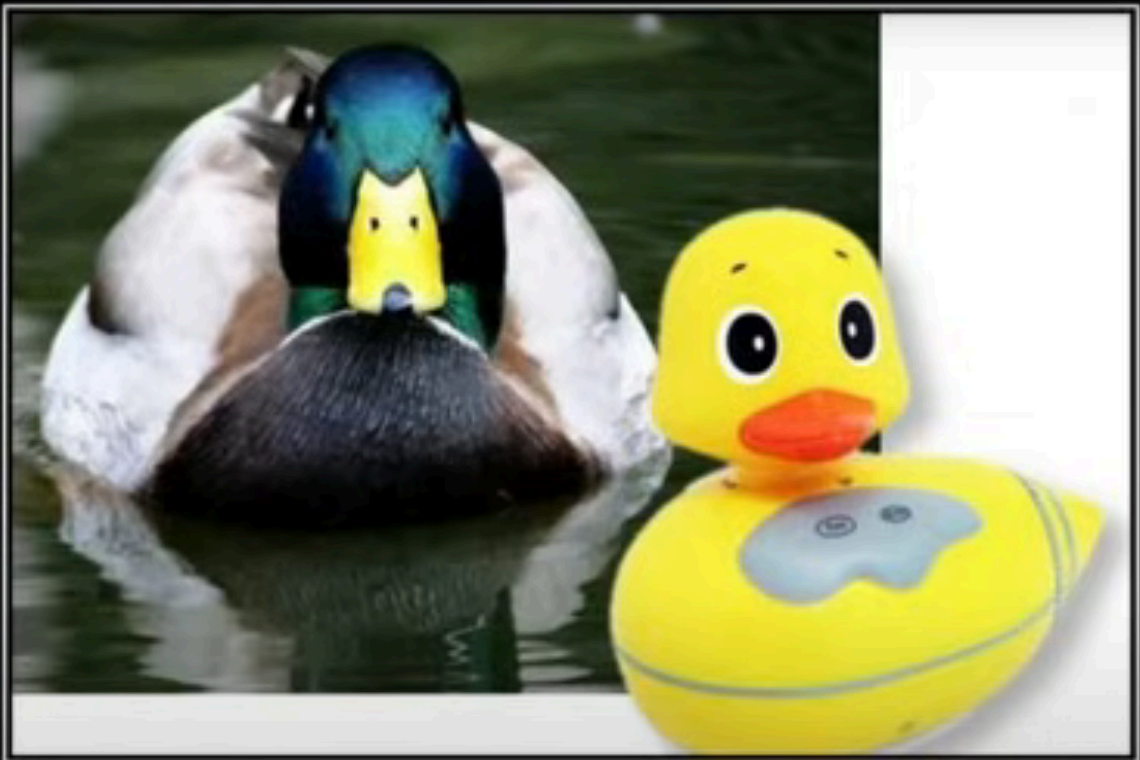
LSP - Liskov Substitution Principle

Sustitución de Liskov

-donde se espera el padre debe funcionar de igual manera si se utiliza al hijo

-el principio viene a desmentir que la POO es una representación de la realidad, cuando nos ajustamos mucho a la realidad seguramente surjan problemas con este principio

Si un método sobrescrito no hace nada o lanza una excepción, es muy probable que estés violando el principio de sustitución de Liskov



LISKOV SUBSTITUTION PRINCIPLE

If It Looks Like A Duck, Quacks Like A Duck, But Needs Batteries - You Probably Have The Wrong Abstraction

Si los tests de la clase padre no funcionan para la hija, también estarás violando este principio.

ISP - Interface Segregation Principle

La problemática surge cuando esas interfaces intentan definir más cosas de las debidas, lo que se denominan fat interfaces.

Si al implementar una interfaz ves que uno o varios de los métodos no tienen sentido y te hace falta dejarlos vacíos o lanzar excepciones

Utilizar el patrón de diseño "Adapter"

-este patrón permite convertir unas interfaces en otras, cosa de que pueda transformar en pequeñas interfaces mejores encapsuladas.

El principio de segregación de interfaces nos ayuda a no obligar a ninguna clase a implementar métodos que no utiliza

-evita errores inesperados o independencias innecesarias, además se reutiliza código de forma mucho más inteligente

DIP - Dependency Inversion Principle

Inversión de Dependencias

-podemos hacer que el código no tenga que depender de factores externos (frameworks usados, base de datos que utilice, etc), solamente utilizar el objeto que ha implementado esa interfaz

Las parte más genérica de nuestro código dependerá por todas partes de detalles de implementación

-no va a ser nada flexible, si cambia algo tendremos que cambiar el core, lógica o el dominio

No quedan claras las dependencias

-va a costar más identificar si rompe otro principio

Es muy complicado hacer tests

-no se puede testear de forma aislada, no se va a saber si es el problema en la clase o de las dependencias

Cualquier instanciación de clases complejas o módulos es una violación de este principio

-lo más común para solucionar esto es utilizar un **Inyector de Dependencias**(es un módulo que trata de instanciar objetos y proveer al resto de objetos que lo necesiten)

Ley de Demeter

Mecanismo de detección de acoplamiento, cuando una clase solamente llama a funciones de otra clase con la cual se relaciona (ninguna que esté fuera o encadenada digamos).

La función solo debe interactuar con su propio ámbito, los objetos que crea o recibe. O con las funciones y estados de esa propia clase, salirse de ahí está incumpliendo la ley.

La Ley de Deméter, también conocida como el principio del mínimo conocimiento, es una guía de diseño en la programación orientada a objetos que busca reducir el acoplamiento entre objetos y promover un código más modular y fácil de mantener. En esencia, sugiere que un objeto solo debe interactuar con sus "amigos inmediatos" y no con objetos a los que accede indirectamente a través de otros.

Encontrarte con muchas llamadas concatenadas

- Incumplimiento de la Ley

- No hay una única forma de resolver esto, es más bien como una alarma para indicar que las relaciones no están bien distribuidas

Añadir métodos extra

- que llamen dentro de cada clase y evitar así el encadenamiento

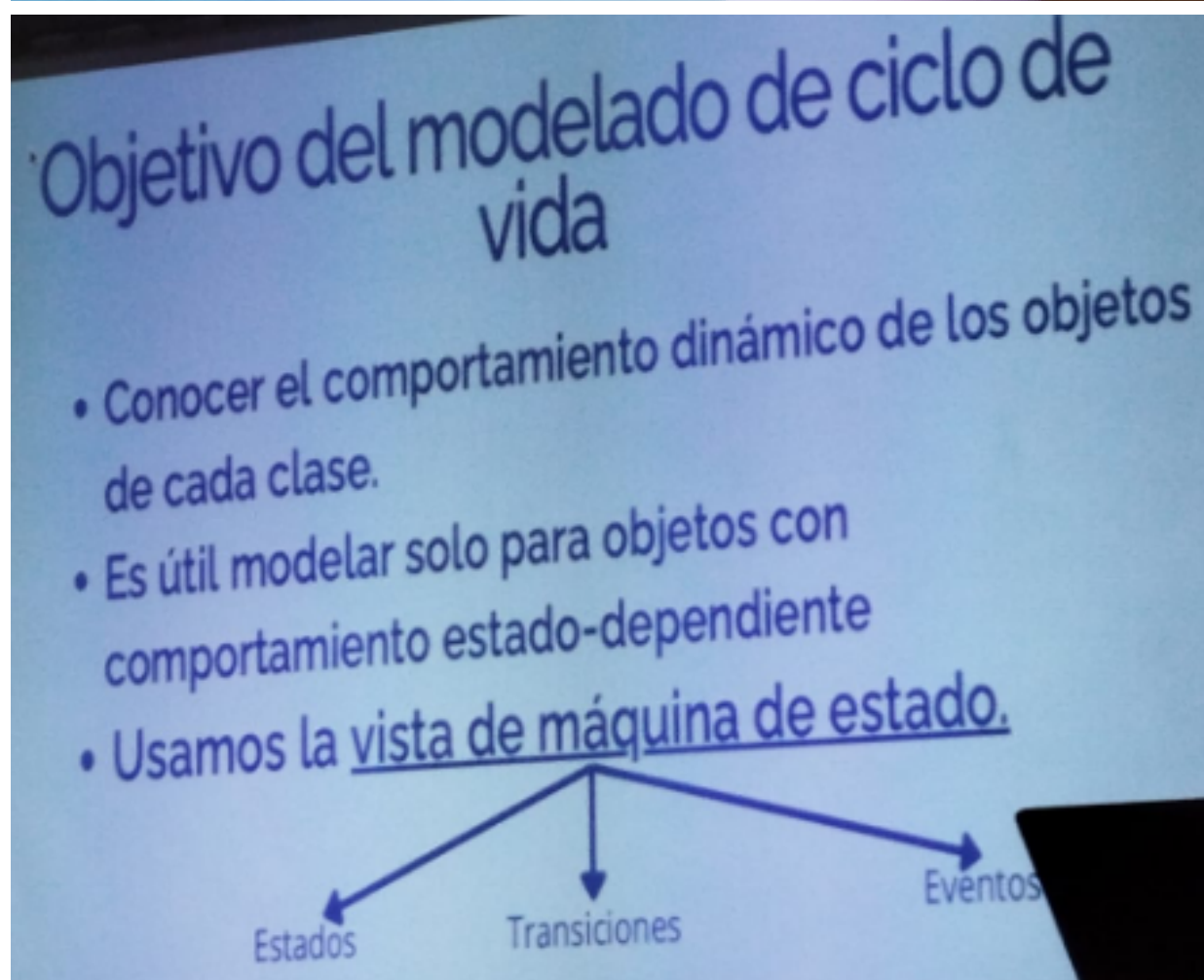
Arquitectura

- una buena arquitectura es muy buena para el desacoplamiento, entonces será mucho más difícil violar esta ley

Comprender mejor tu dominio

- Teniendo en claro nuestro dominio no deberían surgir problemas de este tipo

PRÁCTICA - Ciclo de vida de Objetos



Modulo/Clase/Definición/Objetos.pdf
Página 3 de 4

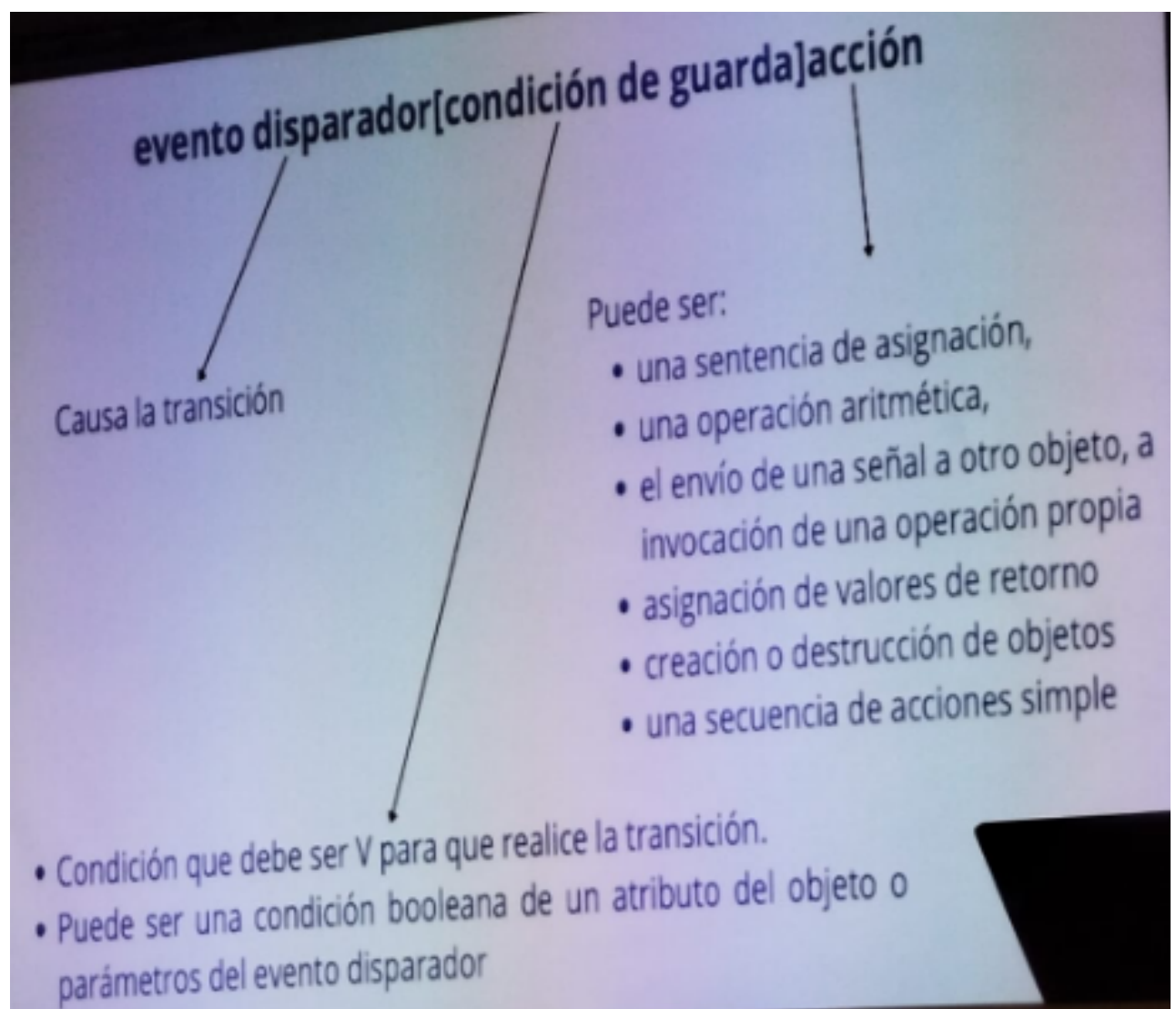
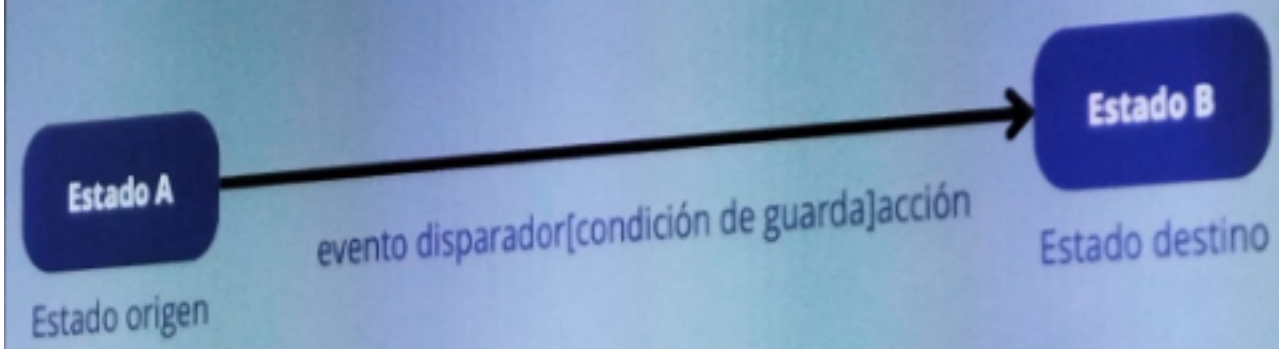
Elementos del Diagrama de Estado

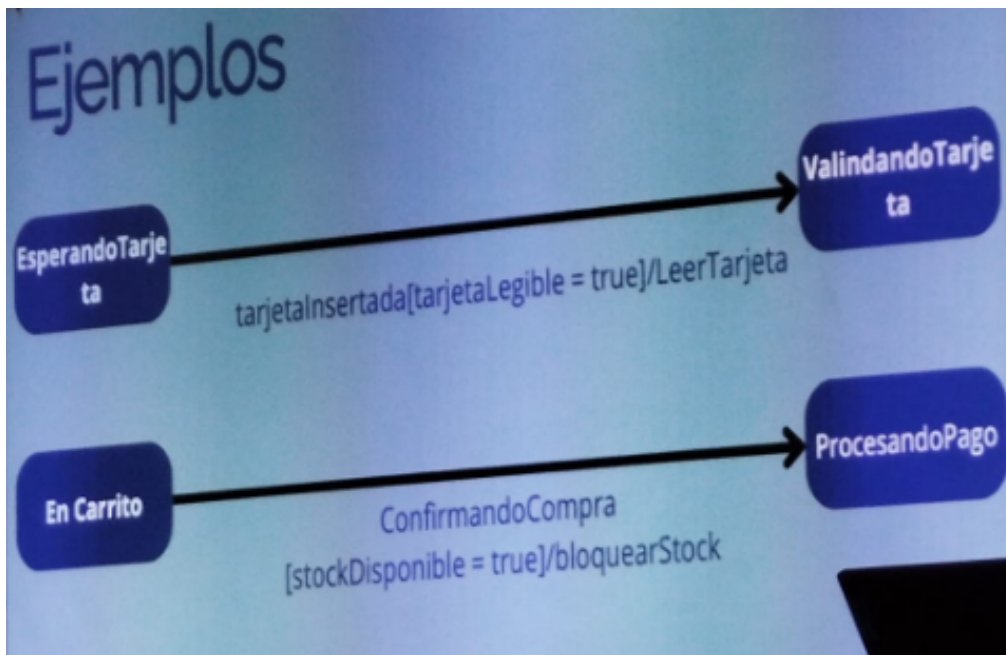
Estado	Transiciones	Eventos
• Condición o situación en un período de tiempo. • Determina su comportamiento.	• Pasaje de un estado a otro.	• Acontecimiento significativo, sin duración, localizado en tiempo y espacio. • Se modela cuando su ocurrencia tiene consecuencias.

Estados

Estado Inicial	Estado Final
	
Puede incluirse un nombre	Puede incluirse un nombre
Puede haber uno SOLO	Puede haber MAS de uno

Transición





Algunos Tips



- Debe haber UN solo estado INICIAL.
- Lo RECOMENDABLE es que haya 1 SOLO estado FINAL.
- Las expresiones de transición SIEMPRE deben estar presente a EXCEPCIÓN de transiciones automáticas y finalización.
- Nombres SIGNIFICATIVOS para los estados.

-transiciones automáticas, no tiene notación. Puede ser el caso de que un estado pasen 3 segundos y automáticamente pase al que sigue (debe estar aclarado en el enunciado)

CLASE 2 - 20/08

The critical design tool for software development is a mind well educated in design principles.

PATRONES GRASP



General Responsibility Assignment Software Patterns

PATRONES GRASP

- Son una serie de buenas prácticas o principios para guiar en la correcta asignación de responsabilidades a las clases.
- General Responsibility Assignment Software Patterns

APPLYING UML AND PATTERNS

An Introduction to Object-Oriented Analysis and Design and Iterative Development



CRAIG LARMAN

Larman, C. (2004). Applying UML and patterns: an introduction to object-oriented analysis and design and iterative development. Pearson.

PATRONES GRASP

Creador

Experto en
Información

Bajo
Acoplamiento

Controlador

Alta
Cohesión

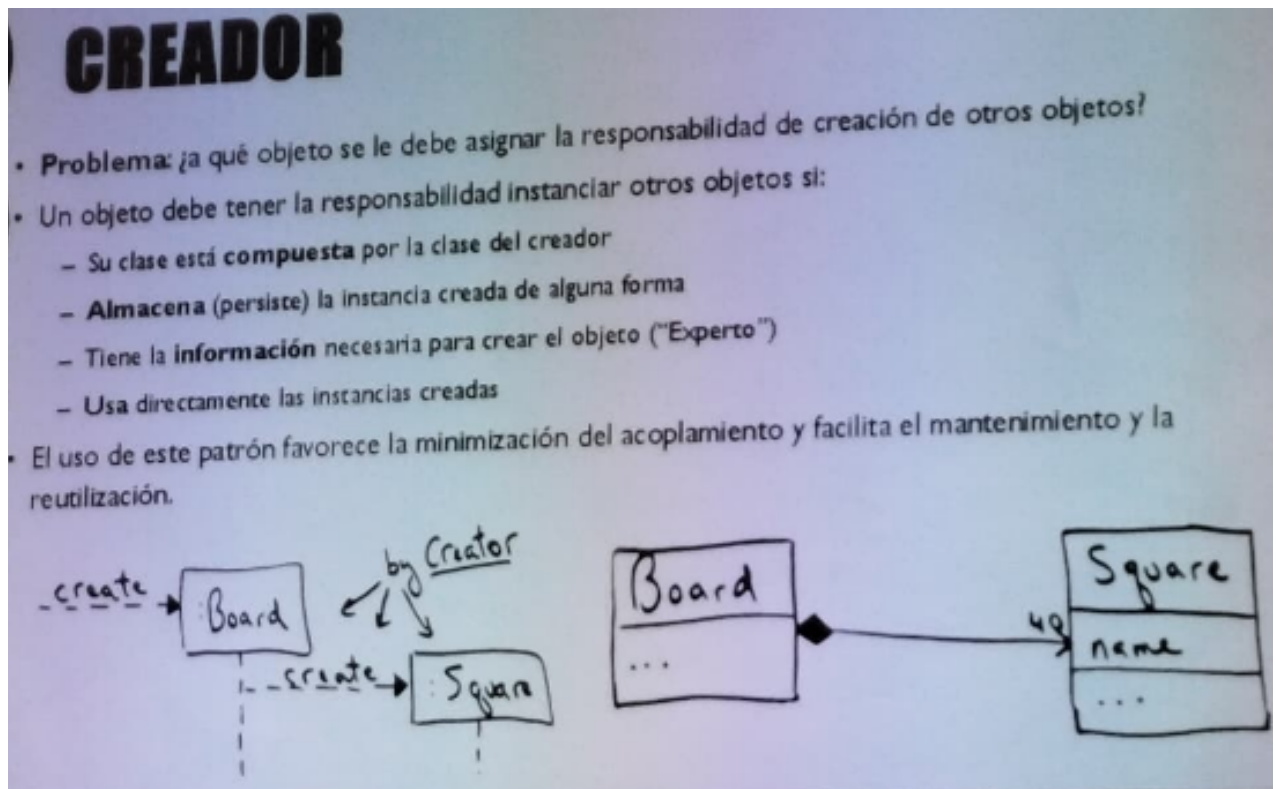
Fabricación
Pura

Indirección

Polimorfismo

Variaciones
protegidas

Creator



- Es fundamental saber a quien le asignamos qué cosa
- Quien tiene las responsabilidades de crear instancias de otros objetos
- Cuando yo tengo una relación todo-parte, la clase que representa al Todo es la mejor opción. Porque reduce el acoplamiento

EJEMPLO

- el creador del objeto es Tablero, pero no confundir con el constructor. Esto indica quién tiene la responsabilidad de llamar al constructor (probablemente esté dentro de Cuadrado)

Experto en Información

- es un patrón más específico que el anterior
- a que clase se le asigna la responsabilidad
- poner responsabilidades que correspondan a otra capa de la arquitectura

Bajo Acoplamiento

- te indica cuan dependiente es una clase de otra
- es una medición subjetiva
- ¿Por qué se busca que una clase esté menos acoplada? sirve para mejorar el mantenimiento y reutilización
- difícil de analizar de forma aislada si hay alto acoplamiento
- favorece la reutilización
- la Herencia también se considera un tipo de acoplamiento

-si se busca tener acoplamiento 0, no va a existir cohesión

Controlador

- no meter en el interfaz, cosas de la aplicación
- intermediario entre el sistema y el usuario
- se encarga de delegar las tareas a los objetos que saben cómo hacerlas

Alta Cohesión

- Cohesión funcional

Fabricación pura

Indirección

Polimorfismo

Variaciones protegidas

(tomi hizo un resumen en la parte de libros sobre GRASP)

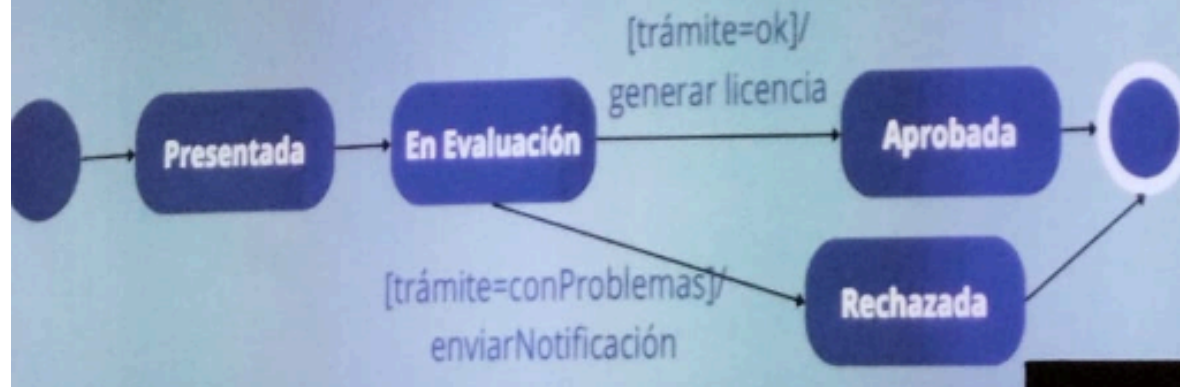
PRÁCTICA - Estados Compuestos

Estados Compuestos

Tomemos como ejemplo ...

La Municipalidad de Ciudad Verde gestiona el trámite para obtener la licencia de conducir. El trámite pasa por las siguientes etapas:

1. *Solicitud presentada*: el ciudadano carga sus datos y entrega la documentación requerida.
2. *Evaluación del trámite*: la municipalidad verifica la información y realiza pruebas para determinar si corresponde otorgar la licencia.
3. *Resolución*: si el trámite fue aprobado, se emite la licencia; si no, se rechaza.



Algunas aclaraciones para el estado "En evaluación"

Dentro de la etapa "En evaluación", el ciudadano debe pasar por varios pasos internos:

- *Verificación de datos: el personal revisa que toda la documentación esté correcta.*
- *Examen teórico: se evalúan los conocimientos de tránsito.*
- *Examen práctico: se prueba la habilidad de conducción.*
- *Evaluación médica: se certifica la aptitud física y visual.*

Una vez que se pasaron por estos pasos y si fueron todos correctos se aprueba la licencia, por cualquiera de los pasos que no se completen o no se cumpla se rechaza la licencia.

Podemos definir un estado
COMPUESTO como

"Es un estado que se ha descompuesto en subestados secuenciales o concurrentes"

- Desde afuera, el sistema solo ve que la licencia está "En evaluación".
- Desde adentro, se modela con más detalle: verificación de datos, los exámenes teóricos y prácticos, y evaluación médica etc.

Los subestados comparten una entrada y salida común (cuando termina la revisión, vuelve al nivel superior).

Entonces ...

